

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – PSEUDOCODE WRITING TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5 – Precision (BL4)	Assessment:	Pseudocode Writing task
Weightage:	30%	Total Marks:	100
CO5 Statement:	Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.		
Student Name:	Vishwa.U	Reg. No.:	192511192
Section / Batch:	Slot-B-2025	Date:	27.08.2026

Instructions to Students

- Identify the relevant concepts of DMA-based data transfer, vectored interrupts, I/O mapping techniques, bus arbitration, and hybrid I/O communication.
- Analyze the requirements of different I/O devices by considering CPU involvement, interrupt priority, latency, bandwidth, throughput, and transfer speed.
- Apply appropriate I/O techniques such as DMA, interrupt-driven I/O, vectored interrupts, memory-mapped I/O, I/O-mapped I/O, and bus arbitration to develop efficient solutions.
- Develop pseudocode algorithms for DMA data transfer, prioritized interrupt handling, dynamic I/O mapping selection, bus arbitration, and hybrid polling–DMA communication.
- Evaluate the proposed I/O mechanisms by interpreting CPU utilization, data transfer efficiency, interrupt response time, communication performance, resource allocation, and overall system suitability.

QUESTIONS

1. A real-time embedded system continuously receives large blocks of data from a high-speed I/O device. Write pseudocode to design a DMA-based data transfer algorithm that transfers data directly from the I/O device to main memory with minimal CPU involvement. The algorithm should initialize the DMA controller, configure source and destination addresses, specify the transfer size, initiate the transfer, and handle completion and error interrupts.
2. An embedded system is connected to multiple I/O devices, each capable of generating interrupts with different priority levels. Develop pseudocode for an interrupt handling mechanism that identifies and prioritizes vectored interrupts, determines the interrupting device, and dispatches the appropriate interrupt service routine. The algorithm should ensure that high-priority interrupts are serviced efficiently while maintaining proper handling of lower-priority requests.
3. A computer system communicates with different I/O devices having varying latency, bandwidth, and data transfer requirements. Write pseudocode for a dynamic I/O selection mechanism that chooses between memory-mapped I/O and I/O-mapped I/O based on the performance requirements of each device. The algorithm should evaluate latency and bandwidth requirements and select the mapping technique that provides efficient communication.
4. A computer system contains multiple devices that simultaneously request access to high-speed communication interfaces such as PCIe, USB, and Thunderbolt. Design pseudocode for a bus arbitration algorithm that manages competing access requests and allocates the appropriate interface to each device. The algorithm should consider device priority, bandwidth requirements, interface availability, and fairness to ensure efficient utilization of communication resources.
5. A real-time embedded system communicates with both low-speed devices, such as simple sensors, and high-speed devices, such as data acquisition units and storage devices. Write pseudocode for a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupt-driven communication for high-speed devices. The algorithm should identify the device type, select the appropriate I/O technique, initiate data transfer, and handle completion and error conditions while minimizing CPU involvement.

Code of Conduct

I Vishwa.U(192511192) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Marks Evaluation:

Q. No	Problem Understanding (20)	Algorithm Design (30)	Use of Control Structures (20)	Pseudocode Structure (10)	Completeness (20)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/30	/ 20	/ 10	/ 20	/ 100

Course Coordinator Faculty Incharge

Q1. DMA-Based Data Transfer Algorithm

Problem Understanding (Inputs / Outputs / Constraints)

- Inputs: device_id, source address (I/O device buffer), destination address (main memory), transfer_size (bytes).
- Outputs: data block moved into main memory, completion status, error status.
- Constraints: CPU must only initialize/configure the transfer and respond to interrupts — the actual word-by-word transfer must occur without CPU intervention (cycle-stealing/burst mode); both completion and error interrupts must be handled.

Pseudocode

```
BEGIN DMA_Transfer(device_id, source_addr, dest_addr, transfer_size)
    // 1. Check controller availability
    IF DMA_Controller.status != IDLE THEN
        RETURN ERROR "DMA controller busy"
    ENDIF

    // 2. Initialize / configure the DMA controller registers
    DMA_Controller.source_register      = source_addr
    DMA_Controller.destination_register = dest_addr
    DMA_Controller.transfer_count      = transfer_size
    DMA_Controller.mode                 = BLOCK_TRANSFER
    DMA_Controller.device_id           = device_id

    // 3. Enable completion / error interrupts
    ENABLE_INTERRUPT(DMA_COMPLETE_IRQ)
    ENABLE_INTERRUPT(DMA_ERROR_IRQ)

    // 4. Initiate transfer and free the CPU
    DMA_Controller.status = BUSY
    SEND_DREQ(device_id)           // device requests bus
    CPU.grant_bus_to(DMA_Controller) // DACK issued
    CPU.continue_other_tasks()      // minimal CPU involvement

    // 5. Autonomous hardware transfer loop (executed by DMA controller, not CPU)
    WHILE transfer_count > 0 AND DMA_Controller.status == BUSY DO
        WAIT_FOR(DREQ signal)
        TRANSFER_WORD(source_addr, dest_addr)
        source_addr = source_addr + WORD_SIZE
        dest_addr   = dest_addr + WORD_SIZE
        transfer_count = transfer_count - 1
        DMA_Controller.transfer_count = transfer_count

        IF BUS_ERROR_DETECTED() THEN
            DMA_Controller.status = ERROR
            TRIGGER_INTERRUPT(DMA_ERROR_IRQ)
            BREAK
        ENDIF
    ENDWHILE

    IF transfer_count == 0 THEN
        DMA_Controller.status = COMPLETE
        TRIGGER_INTERRUPT(DMA_COMPLETE_IRQ)
```

```

    ENDIF
END

// --- Interrupt Service Routines (CPU side) ---
ISR DMA_Complete_Handler()
    SAVE_CONTEXT()
    LOG("DMA transfer completed successfully")
    DMA_Controller.status = IDLE
    NOTIFY_PROCESS(device_id, SUCCESS)
    RESTORE_CONTEXT()
    RETURN_FROM_INTERRUPT()
END

ISR DMA_Error_Handler()
    SAVE_CONTEXT()
    error_code = DMA_Controller.error_register
    LOG("DMA transfer failed: " + error_code)
    retry_count = retry_count + 1
    IF retry_count < MAX_RETRIES THEN
        DMA_Transfer(device_id, source_addr, dest_addr, remaining_count) // retry
    ELSE
        DMA_Controller.status = IDLE
        NOTIFY_PROCESS(device_id, FAILURE, error_code)
    ENDIF
    RESTORE_CONTEXT()
    RETURN_FROM_INTERRUPT()
END

```

Completeness Notes

- Handles the happy path (full transfer, IRQ on completion), the busy-controller case, bus errors mid-transfer, and automatic bounded retries on error.
- CPU involvement is limited to setup and interrupt servicing only — satisfies the 'minimal CPU involvement' requirement.

Q2. Vectored Interrupt Handling with Priority

Problem Understanding (Inputs / Outputs / Constraints)

- Inputs: active IRQ lines from multiple devices, each device's priority level and interrupt-vector address.
- Outputs: correct ISR dispatched for the interrupting device, in priority order.
- Constraints: high-priority interrupts must be serviced promptly (including pre-empting a lower-priority ISR already running); lower-priority requests must not be lost — they are queued, not dropped.

Pseudocode

```
STRUCTURE InterruptRequest
    device_id, priority_level, vector_address    // priority_level: 0 = highest
END STRUCTURE

PriorityQueue pending_interrupts = EMPTY
CPU.current_priority_mask = LOWEST

BEGIN Interrupt_Controller_Main()
    WHILE SYSTEM_RUNNING DO
        FOR EACH device IN CONNECTED_DEVICES DO
            IF device.IRQ_line == ACTIVE THEN
                req = NEW InterruptRequest(device.id, device.priority,
device.vector_addr)
                pending_interrupts.INSERT_SORTED_BY_PRIORITY(req)
            ENDIF
        ENDFOR
        Process_Interrupts()
    ENDWHILE
END

BEGIN Process_Interrupts()
    WHILE pending_interrupts NOT EMPTY DO
        current = pending_interrupts.PEEK_HIGHEST_PRIORITY()

        IF CPU.interrupts_enabled AND current.priority > CPU.current_priority_mask
THEN
            pending_interrupts.REMOVE(current)
            SAVE_CONTEXT(CPU.registers, CPU.PC)
            previous_mask = CPU.current_priority_mask
            CPU.current_priority_mask = current.priority    // mask same/lower priority

            vector      = READ_VECTOR_TABLE(current.vector_address)
            ISR_address = vector.ISR_pointer
            Dispatch_ISR(current.device_id, ISR_address)

            CPU.current_priority_mask = previous_mask
            RESTORE_CONTEXT()
        ELSE
            BREAK    // nothing pending outranks what is currently executing
        ENDIF
    ENDWHILE
```

```

END

BEGIN Dispatch_ISR(device_id, ISR_address)
    JUMP_TO(ISR_address)
    SWITCH device_id
        CASE DISK_CONTROLLER : CALL Disk_ISR()
        CASE NETWORK_CARD    : CALL Network_ISR()
        CASE TIMER            : CALL Timer_ISR()
        CASE KEYBOARD         : CALL Keyboard_ISR()
        DEFAULT               : CALL Generic_ISR(device_id)
    END SWITCH
    ACKNOWLEDGE_INTERRUPT(device_id)    // clear the IRQ line
END

// Pre-emption: a new, higher-priority interrupt arrives while an ISR is executing
ON NEW_INTERRUPT_DURING_ISR(new_request)
    IF new_request.priority > CPU.current_priority_mask THEN
        SAVE_CONTEXT()                // nest: push current ISR's state
        Process_Interrupts()          // service the higher-priority request now
        RESTORE_CONTEXT()             // resume the pre-empted ISR
    ELSE
        pending_interrupts.INSERT_SORTED_BY_PRIORITY(new_request) // queued, not
dropped
    ENDIF
END

```

Completeness Notes

- Covers simultaneous interrupts (sorted queue), nested/pre-emptive servicing of higher priority interrupts arriving mid-ISR, and safe queuing (never dropping) of lower-priority requests.

Q3. Dynamic Selection of Memory-Mapped vs I/O-Mapped I/O

Problem Understanding (Inputs / Outputs / Constraints)

- Inputs: per-device latency requirement, bandwidth requirement, data-block size, available memory address space.
- Outputs: mapping decision (Memory-Mapped I/O or I/O-Mapped I/O) and corresponding address assignment for each device.
- Constraints: decision must be driven by measured/declared latency and bandwidth thresholds, and must fall back gracefully if memory address space is limited.

Pseudocode

```
CONSTANT LATENCY_THRESHOLD = 10    // microseconds
CONSTANT BANDWIDTH_THRESHOLD = 100  // MB/s

STRUCTURE Device
    id, latency_requirement, bandwidth_requirement, needs_large_block
END STRUCTURE

BEGIN Select_IO_Mapping(device)
    IF device.latency_requirement < 0 OR device.bandwidth_requirement < 0 THEN
        RETURN ERROR "Invalid device parameters"
    ENDIF

    is_low_latency   = (device.latency_requirement <= LATENCY_THRESHOLD)
    is_high_bandwidth = (device.bandwidth_requirement >= BANDWIDTH_THRESHOLD)

    // Decision logic
    IF is_high_bandwidth OR device.needs_large_block == TRUE THEN
        mapping = MEMORY_MAPPED_IO    // wide bus, burst access, large address
range
    ELSE IF is_low_latency AND NOT is_high_bandwidth THEN
        mapping = IO_MAPPED_IO        // simple/control devices, isolated address
space
    ELSE
        mapping = IO_MAPPED_IO        // default for low-bandwidth, non-critical
devices
    ENDIF

    // Constraint check: fall back if memory-mapped space is exhausted
    IF mapping == MEMORY_MAPPED_IO AND MEMORY_ADDRESS_SPACE_AVAILABLE() == FALSE THEN
        LOG_WARNING("Insufficient memory-mapped space; falling back to I/O-mapped")
        mapping = IO_MAPPED_IO
    ENDIF

    // Configure device for the chosen technique
    IF mapping == MEMORY_MAPPED_IO THEN
        ASSIGN_MEMORY_ADDRESS_RANGE(device.id)
        CONFIGURE_FOR_LOAD_STORE_ACCESS(device.id)
    ELSE
        ASSIGN_IO_PORT_ADDRESS(device.id)
```



```
        CONFIGURE_FOR_IN_OUT_INSTRUCTIONS(device.id)
    ENDIF

    RETURN mapping
END

BEGIN Dynamic_IO_Manager(device_list)
    FOR EACH device IN device_list DO
        result = Select_IO_Mapping(device)
        LOG("Device " + device.id + " -> " + result)
        REGISTER_DEVICE_MAPPING(device.id, result)
    ENDFOR
END
```

Completeness Notes

- Handles invalid input, the address-space-exhaustion edge case with graceful fallback, and processes an arbitrary list of heterogeneous devices in one pass.

Q4. Bus Arbitration Algorithm (PCIe / USB / Thunderbolt)

Problem Understanding (Inputs / Outputs / Constraints)

- Inputs: competing device requests, each with priority, bandwidth need, requested interface type (PCIe/USB/Thunderbolt), and time already waited.
- Outputs: bus/interface grants issued to devices, one at a time per interface, respecting available bandwidth.
- Constraints: must weigh device priority AND bandwidth AND interface availability, and must remain fair — no request should starve indefinitely.

Pseudocode

```
STRUCTURE BusRequest
    device_id, priority, bandwidth_needed, interface_type, wait_time
END STRUCTURE

Queue request_queue = EMPTY
MAP interface_status[PCIe, USB, Thunderbolt] = AVAILABLE
CONSTANT STARVATION_LIMIT = 5 // cycles before priority is boosted

BEGIN Bus_Arbitration_Manager()
    WHILE SYSTEM_RUNNING DO
        FOR EACH device IN REQUESTING_DEVICES DO
            IF device.bus_request_line == ACTIVE THEN
                req = NEW BusRequest(device.id, device.priority,
                                     device.bandwidth_needed,
                                     device.interface_type, 0)
                request_queue.ADD(req)
            ENDIF
        ENDFOR

        // Aging: prevent starvation of long-waiting requests
        FOR EACH req IN request_queue DO
            req.wait_time = req.wait_time + 1
            IF req.wait_time > STARVATION_LIMIT THEN
                req.priority = req.priority + 1
            ENDIF
        ENDFOR

        Grant_Bus_Access()
    ENDWHILE
END

BEGIN Grant_Bus_Access()
    SORT request_queue BY priority DESC, bandwidth_needed DESC, wait_time DESC

    FOR EACH req IN request_queue DO
        iface = req.interface_type

        IF interface_status[iface] == AVAILABLE THEN
            IF Check_Bandwidth_Capacity(iface, req.bandwidth_needed) == TRUE THEN
                interface_status[iface] = BUSY
                GRANT_ACCESS(req.device_id, iface)
            ENDIF
        ENDIF
    ENDFOR
END
```

```

        START_TRANSACTION_TIMER(req.device_id, iface)
        request_queue.REMOVE(req)
    ELSE
        LOG("Insufficient bandwidth on " + iface + "; " + req.device_id + "
stays queued")
    ENDIF
    ELSE IF PREEMPTION_ALLOWED AND req.priority > CURRENT_HOLDER_PRIORITY(iface)
THEN
        PREEMPT_CURRENT_TRANSACTION(iface)
        interface_status[iface] = AVAILABLE
        GRANT_ACCESS(req.device_id, iface)
        request_queue.REMOVE(req)
    ELSE
        CONTINUE    // interface busy; wait this cycle
    ENDIF
ENDFOR
END

ON TRANSACTION_COMPLETE(device_id, iface)
    interface_status[iface] = AVAILABLE
    STOP_TRANSACTION_TIMER(device_id)
    RELEASE_BUS(device_id)
END

BEGIN Check_Bandwidth_Capacity(iface, requested_bw)
    available_bw = INTERFACE_MAX_BANDWIDTH[iface] - CURRENT_USED_BANDWIDTH[iface]
    RETURN (requested_bw <= available_bw)
END

```

Completeness Notes

- Combines priority-based selection with bandwidth-capacity checks, optional pre-emption, and an explicit aging mechanism so fairness is guaranteed even under sustained high-priority traffic.

Q5. Hybrid Polling–DMA I/O Communication

Problem Understanding (Inputs / Outputs / Constraints)

- Inputs: a mixed device list (low-speed sensors and high-speed DAQ/storage units) with a speed rating and data size for each.
- Outputs: data transferred from each device via the technique appropriate to its speed class; completion/error status.
- Constraints: must classify each device, poll only low-speed devices, and use DMA + interrupt only for high-speed devices, while keeping overall CPU involvement minimal.

Pseudocode

```
CONSTANT SPEED_THRESHOLD = 10    // MB/s : below this => low-speed (poll)

STRUCTURE IODevice
    id, type, speed_rating, data_size, buffer_address
END STRUCTURE

BEGIN Hybrid_IO_Manager(device_list)
    FOR EACH device IN device_list DO
        IF Classify_Device(device) == LOW_SPEED THEN
            Poll_Device(device)
        ELSE
            DMA_Interrupt_Transfer(device)
        ENDIF
    ENDFOR
END

BEGIN Classify_Device(device)
    IF device.speed_rating < SPEED_THRESHOLD THEN
        RETURN LOW_SPEED
    ELSE
        RETURN HIGH_SPEED
    ENDIF
END

// ---- Low-speed path: Polling ----
BEGIN Poll_Device(device)
    timeout = MAX_POLL_TIME
    WHILE timeout > 0 DO
        status = READ_STATUS_REGISTER(device.id)
        IF status == DATA_READY THEN
            data = READ_DATA_REGISTER(device.id)
            STORE(data, device.buffer_address)
            RETURN SUCCESS
        ELSE IF status == DEVICE_ERROR THEN
            LOG_ERROR("Polling error on " + device.id)
            RETURN FAILURE
        ENDIF
        timeout = timeout - 1
        WAIT(POLL_INTERVAL)
    ENDWHILE
    LOG_ERROR("Polling timeout for " + device.id)
    RETURN TIMEOUT_FAILURE
```

```

END

// ---- High-speed path: DMA + Interrupt ----
BEGIN DMA_Interrupt_Transfer(device)
    IF DMA_Controller.status != IDLE THEN
        QUEUE_REQUEST(device)      // defer until controller free
        RETURN
    ENDIF

    DMA_Controller.source          = device.buffer_address
    DMA_Controller.destination     = MAIN_MEMORY_ADDRESS
    DMA_Controller.count           = device.data_size
    ENABLE_INTERRUPT(DMA_COMPLETE_IRQ)
    ENABLE_INTERRUPT(DMA_ERROR_IRQ)
    DMA_Controller.status = BUSY
    START_DMA_TRANSFER()
    CPU.free_for_other_tasks()      // CPU not blocked while DMA runs
END

ISR DMA_Complete_Handler(device_id)
    SAVE_CONTEXT()
    DMA_Controller.status = IDLE
    NOTIFY_APPLICATION(device_id, SUCCESS)
    IF QUEUE_NOT_EMPTY() THEN
        next_device = DEQUEUE_REQUEST()
        DMA_Interrupt_Transfer(next_device)
    ENDIF
    RESTORE_CONTEXT()
END

ISR DMA_Error_Handler(device_id)
    SAVE_CONTEXT()
    LOG_ERROR("DMA error on " + device_id)
    DMA_Controller.status = IDLE
    retry_count[device_id] = retry_count[device_id] + 1
    IF retry_count[device_id] < MAX_RETRIES THEN
        DMA_Interrupt_Transfer(device_id)
    ELSE
        NOTIFY_APPLICATION(device_id, FAILURE)
    ENDIF
    RESTORE_CONTEXT()
END

```

Completeness Notes

- Explicitly branches by device classification, queues high-speed requests when the DMA controller is busy (no data loss), and retries on error up to a bounded limit — covering normal, contention, and failure cases for both device classes.